

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[Sign In](#)[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOO](#) [›](#)

Python Tutorial

[Python HOME](#)[Python Intro](#)[Python Get Started](#)[Python Syntax](#)[Python Comments](#)[Python Variables](#)[Python Data Types](#)[Python Numbers](#)[Python Casting](#)[Python Strings](#)[Python Booleans](#)[Python Operators](#)[Python Lists](#)[Python Tuples](#)[Python Sets](#)[Python Dictionaries](#)[Python If...Else](#)[Python Match](#)[Python While Loops](#)[Python For Loops](#)[Python Functions](#)[Python Decorators](#)[Python Range](#)[Python Lambda](#)[Python Arrays](#)[Python OOP](#)[Python Classes/Objects](#)[Python Inheritance](#)[Python Iterators](#)[Python Polymorphism](#)[Python Scope](#)[Python Modules](#)

Python Operators

[◀ Previous](#)[Next ▶](#)

Python Operators

Operators are used to perform operations on variables and values.

In the example below, we use the **+** operator to add together two values:

Example

[Get your own Python Server](#)

```
print(10 + 5)
```

[Run example »](#)

Python divides the operators in the following groups:

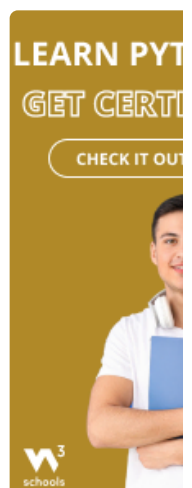
- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example	Try it
----------	------	---------	--------

ADVERTISEMENT

[COLOR PICKER](#)

+	Addition	$x + y$	Try it »
-	Subtraction	$x - y$	Try it »
*	Multiplication	$x * y$	Try it »
/	Division	x / y	Try it »
%	Modulus	$x \% y$	Try it »
**	Exponentiation	$x ** y$	Try it »
//	Floor division	$x // y$	Try it »


[AI](#)

Python Assignment Operators

Assignment operators are used to assign values to variables:

Operator	Example	Same As	Try it
=	$x = 5$	$x = 5$	Try it »
+=	$x += 3$	$x = x + 3$	Try it »
-=	$x -= 3$	$x = x - 3$	Try it »
*=	$x *= 3$	$x = x * 3$	Try it »
/=	$x /= 3$	$x = x / 3$	Try it »
%=	$x \% = 3$	$x = x \% 3$	Try it »
//=	$x //= 3$	$x = x // 3$	Try it »
**=	$x ** = 3$	$x = x ** 3$	Try it »
&=	$x \& = 3$	$x = x \& 3$	Try it »
=	$x = 3$	$x = x 3$	Try it »
^=	$x \wedge = 3$	$x = x \wedge 3$	Try it »
>>=	$x >> = 3$	$x = x >> 3$	Try it »
<<=	$x << = 3$	$x = x << 3$	Try it »
:=	<code>print(x := 3)</code>	$x = 3$ <code>print(x)</code>	Try it »

[R](#)

ADVERTISEMENT

[REMOVE ADS](#)

Python Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example	Try it
==	Equal	x == y	Try it »
!=	Not equal	x != y	Try it »
>	Greater than	x > y	Try it »
<	Less than	x < y	Try it »
>=	Greater than or equal to	x >= y	Try it »
<=	Less than or equal to	x <= y	Try it »

Python Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example	Try it
and	Returns True if both statements are true	x < 5 and x < 10	Try it »
or	Returns True if one of the statements is true	x < 5 or x < 4	Try it »
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)	Try it »

Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example	Try it
is	Returns True if both variables are the same object	x is y	Try it »
is not	Returns True if both variables are not the same object	x is not y	Try it »

Python Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example	Try it
in	Returns True if a sequence with the specified value is present in the object	x in y	Try it »
not in	Returns True if a sequence with the specified value is not present in the object	x not in y	Try it »

Python Bitwise Operators

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example	Try it
&	AND	Sets each bit to 1 if both bits are 1	x & y	Try it »
	OR	Sets each bit to 1 if one of two bits is 1	x y	Try it »
^	XOR	Sets each bit to 1	x ^ y	Try it »

		if only one of two bits is 1		
~	NOT	Inverts all the bits	~x	Try it »
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2	Try it »
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2	Try it »

Operator Precedence

Operator precedence describes the order in which operations are performed.

Example

Parentheses has the highest precedence, meaning that expressions inside parentheses must be evaluated first:

```
print((6 + 3) - (6 + 3))
```

[Run example »](#)

Example

Multiplication `*` has higher precedence than addition `+`, and therefore multiplications are evaluated before additions:

```
print(100 + 5 * 3)
```

[Run example »](#)

The precedence order is described in the table below, starting with the highest precedence at the top:

Operator	Description	Try it
<code>()</code>	Parentheses	Try it »
<code>**</code>	Exponentiation	Try it »
<code>+x</code> <code>-x</code> <code>~x</code>	Unary plus, unary minus, and bitwise NOT	Try it »
<code>*</code> <code>/</code> <code>//</code> <code>%</code>	Multiplication, division, floor division, and modulus	Try it »
<code>+</code> <code>-</code>	Addition and subtraction	Try it »
<code><<</code> <code>>></code>	Bitwise left and right shifts	Try it »
<code>&</code>	Bitwise AND	Try it »
<code>^</code>	Bitwise XOR	Try it »
<code> </code>	Bitwise OR	Try it »
<code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code> <code>is</code> <code>is</code> <code>not in</code> <code>not in</code>	Comparisons, identity, and membership operators	Try it »
<code>not</code>	Logical NOT	Try it »
<code>and</code>	AND	Try it »
<code>or</code>	OR	Try it »

If two operators have the same precedence, the expression is evaluated from left to right.

Example

Addition `+` and subtraction `-` has the same precedence, and therefore we evaluate the expression from left to right:

```
print(5 + 4 - 7 + 3)
```

[Run example »](#)